

SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL



PADRÕES DE UTILIZAÇÃO DO GITHUB

CLARISSON HENRIQUE DE PAIVA BENTO – 4005

MANOEL AUGUSTO ALVES DA COSTA - 5379

Florestal - MG

Setembro de 2025

Para o desenvolvimento do **BitTint**, optamos por um fluxo de *branches* simplificado, dividido em dois eixos principais: um para código, utilizando a *branch develop*, e outro para documentos, utilizando a *branch documents*. Essa escolha tem como objetivo garantir organização, rastreabilidade e controle de versões durante o projeto.

Branches

As *branches* utilizadas se dividem em fixas e temporárias. As fixas são *main*, *develop* e *documents*, enquanto as temporárias são *feature/** e *docs/**. As *branches* temporárias existem apenas enquanto a tarefa ou documento está em andamento e são fechadas após o *merge*.

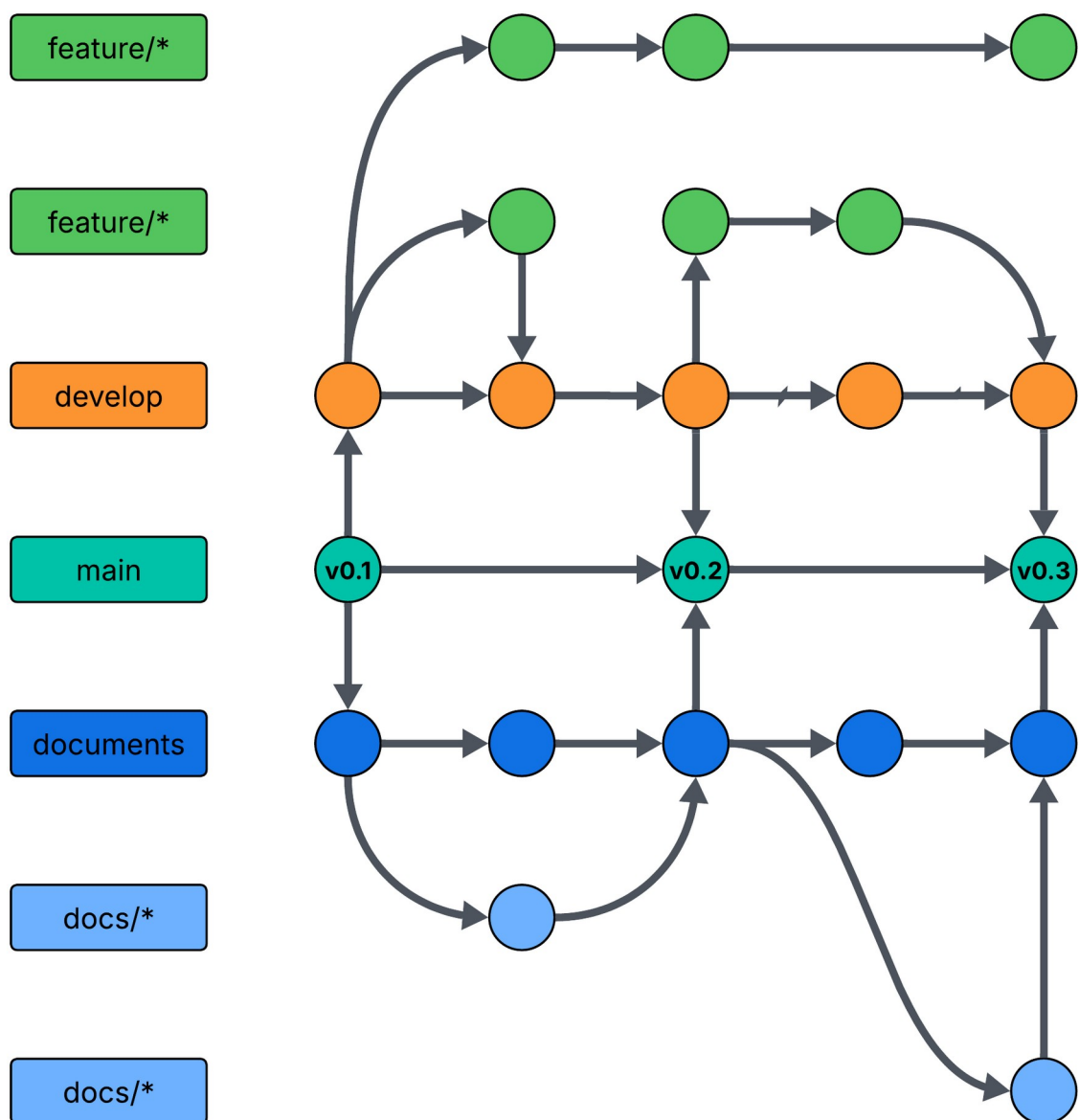


Figura 1: Fluxo de branches para alunos Sêniores

Branch Main

A *branch main* é a principal do repositório. Ela contém tanto o código quanto os documentos validados e serve como referência estável do projeto. Todo *merge* aceito na *main* deve receber uma *tag* de versão, como v0.1, para facilitar o rastreamento. Apenas as *branches develop* e *documents* podem interagir com a *main*, sempre no final da *sprint*. Diferente do GitFlow tradicional, não utilizaremos *branches* de *hotfix*, portanto, *bugs* identificados na *main* só serão corrigidos na próxima versão.

Branch Develop

É a *branch* onde se mantém o código da próxima versão do projeto. Ela funciona como uma cópia da *branch* principal, mas com funcionalidades (*features*) em desenvolvimento que ainda não foram publicadas. Ao final da *sprint*, essas alterações são integradas à *branch main* por meio de um *merge*. Dessa forma, a *develop* serve como base para a criação e evolução de novas *features*.

Branch Documents

A *branch documents* tem função semelhante à *develop*, mas dedicada exclusivamente ao material de apoio e documentação do projeto. Nela ficam armazenados arquivos de texto, manuais, atas, relatórios, protótipos e quaisquer outros registros e artefatos relacionados a desenvolvimento. Assim como ocorre na *develop*, novas alterações são realizadas em *branches* derivadas e depois integradas à *documents* por meio de *merges*, garantindo que a documentação evolua de forma organizada e paralela ao código-fonte.

Branch Feature

As *branches feature* são temporárias e auxiliares dentro do fluxo de desenvolvimento, utilizadas para implementar funcionalidades específicas. Adotaremos uma convenção de nomenclatura para todas as *branches* criadas ao longo do projeto:

- *feature/nome_do_recurso*
- **Juniões** não devem usar a *branch feature/**.

Essas *branches* devem sempre ser criadas a partir da *branch develop*. Quando a funcionalidade estiver concluída e o *pull request* aprovado, a *branch feature* em questão será deletada. Se houver dez funcionalidades a serem desenvolvidas, devem ser criadas dez

branches independentes. É importante destacar que as *branches feature* nunca interagem diretamente com a *main*, apenas com a *develop*.

Branch Docs

As *branches docs/** são temporárias e seguem um fluxo semelhante às *branches feature/**, mas voltadas exclusivamente para a produção e atualização de documentos do projeto. Elas devem sempre ser criadas a partir da *branch documents* e seguem uma convenção de nomenclatura:

- *docs/nome_do_documento*
- **Juniore**s não devem usar as *branches documents* e *docs/**.

Após a conclusão do documento e a aprovação do *pull request*, a *branch docs* será removida. Assim como nas *branches* de código, cada documento em desenvolvimento deve ter sua própria *branch* independente.

Branch Junior

As *branches junior/** também são temporárias e têm função análoga às *branches feature/**, mas voltadas exclusivamente para entrega e organização das tarefas realizadas pelos alunos juniores. Elas devem sempre ser criadas a partir da *branch develop* e seguem uma convenção de nomenclatura:

- *junior/nome_do_aluno/nome_do_recurso*

Após a conclusão da tarefa e a aprovação do *pull request*, a *branch junior/** será removida. Juniores devem criar **apenas** *branchs junior/**, não sendo necessário interagir com *branches feature/** e *docs/**.

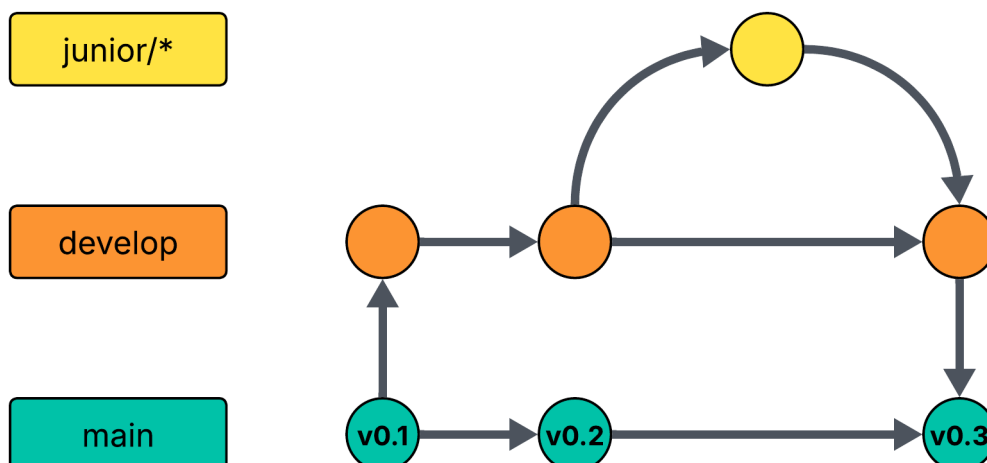


Figura 2: Fluxo de branches para alunos Juniores.

Commit e Pull Request

Padrão de *Commit*

O *commit* é o ponto de partida do seu trabalho no repositório. Para garantir que o histórico de alterações seja claro e fácil de rastrear, adote um padrão de título para cada tipo de mudança. O título deve ser conciso e direto, descrevendo a essência do trabalho executado.

- **:sparkles: feat:** Usado para novas funcionalidades.
- **:books: docs:** Usado para novos documentos ou atualizações em documentos existentes.
- **:bug: fix:** Usado para correções de *bugs*.
- **:recycle: refactor:** Usado para refatorar códigos, ou seja, modificá-los sem alterar sua funcionalidade.
- **:star2: style:** Usado para alterações de estilização que não afetam o funcionamento ou a performance do código.

Exemplos:

- **:sparkles: feat:** Adiciona funcionalidade de login
- **:books: docs:** Cria documento de casos de uso

Padrão de *Pull Request*

Quando a sua tarefa estiver concluída (seja um documento ou uma funcionalidade), o próximo passo é solicitar a integração do seu código na *branch* principal. Isso é feito por meio de um *pull request* (PR), que serve para descrever e solicitar a aprovação das suas alterações.

Recomendamos que você abra o PR pela interface do GitHub para aproveitar recursos como *labels* e *milestones*, que são cruciais para a rastreabilidade. Lembre-se de designar os analistas de qualidade responsáveis pela revisão nos campos *Reviewers* e *Assignees*.

Como Criar um *Pull Request*

Antes de confirmar o PR, é fundamental verificar a *branch* de destino correta. Na tela de criação, observe os campos:

- **base:** representa a *branch* de destino, ou seja, para onde você quer que suas alterações sejam integradas.

- **compare:** representa a *branch* de origem, de onde vêm as suas alterações.

Exemplo:

- **base:** develop ← **compare:** feature/login
- **base:** documents ← **compare:** docs/casos_de_uso
- **base:** develop ← **compare:** junior/rafael/classe_aluno

Conteúdo do Pull Request

- **Atenção:** Você não deve replicar o padrão de *commit* para o *pull request*.
- **Título:** Indique de forma resumida o propósito da *branch* e a funcionalidade ou documento a que se refere. **Não** use “feat”, “fix”, “docs” e semelhantes para o PR.
- **Descrição:** Forneça um resumo geral do que foi realizado na *branch*. Inclua informações sobre arquivos criados ou modificados, além de trechos de código importantes. Não é necessário detalhar todas as alterações; essa informação já deve constar nos *commits*.

Processo de Análise

Após a abertura do PR, o analista de qualidade responsável irá analisar as suas alterações. Se a solicitação não atender aos requisitos estabelecidos, ele abrirá uma *issue* detalhando os pontos que precisam ser corrigidos. Uma vez que as correções sejam feitas, o PR pode ser aprovado e, finalmente, integrado à *branch* de destino.

Labels

Labels são *tags* usadas para categorizar *issues* e *pull requests*. As seguintes *labels* foram criadas no GitHub:

- **ajuda necessária:** precisa de suporte ou você não entendeu como fazer a tarefa
- **análise de qualidade:** precisa de validação de qualidade
- **análise de código:** precisa de validação de um Dev. Sênior
- **bug:** algo não está funcionando como deveria
- **documento:** correções ou alterações em documentos
- **design:** correções ou alterações em artefatos de *design*
- **duplicado:** essa *issue* ou *pull request* já existem
- **feature:** nova *feature* ou melhoria em uma *feature* existente

- **fix:** corrige um *bug* ou mal funcionamento de uma *feature*
- **infraestrutura:** CI/CD, *build*, *config*, etc...
- **junior:** Precisa da atenção do desenvolvedor sênior

Milestones

As *milestones* representam um tipo de *tag* também, mas essas servem para nomear cada *sprint* do projeto. Toda *issue* ou *pull request* deve estar vinculada à *milestone* correspondente, permitindo assim o acompanhamento do progresso da *sprint*.

Dúvidas

Em caso de dúvidas sobre este documento ou sobre o fluxo do GitHub, deve-se entrar em contato pelo Discord do projeto, marcando o Gerente de Configuração ou os Analistas de Qualidade, responsáveis por definirem essas regras.